Tile Engine ↔ Dispatcher Parity

Basic usage

Where you are

All commands run from `dispatcher/parity/`. This box has python3, g++, and hipcc but **no GPU** and **no cmake**, so GPU stages auto-skip here and run on a GPU node unchanged.

What works without a GPU

- Translating a TE config to dispatcher configs.
- **Identifier parity** — the main offline↔runtime guarantee.
- Driving codegen for one config (emits the header).
- **Building** the harness with hipcc (running it needs a GPU).
- `check_parity.py` Stage 1; Stages 2–3 report SKIPPED.

Prerequisites

```
python3 # 3.8+
g++ (or c++/clang++) # for the C++ identifier oracle
hipcc # only to BUILD the harness (deliverable d)
a ROCm GPU node # only to RUN numerical / performance stages
```

1. Identifier parity (CPU, fast)

The one to run anywhere. Compiles the C++ oracle on first use.

```
python check_identifier_parity.py configs/single_fp16_rcr.json
python check_identifier_parity.py configs/single_fp16_rcr.json --verbose
```

Expected tail:

```
identifier parity: 1/1 configs match
(python encode_identifier vs C++ KernelKey::encode_identifier)
```

2. Generate one kernel header

```
python drive_codegen.py configs/single_fp16_rcr.json --index 0

# see the codegen command without running it:
python drive_codegen.py configs/single_fp16_rcr.json --dry-run
```

Writes `generated/parity_single/gemm_<name>.hpp` and prints the expected registry identifier.

3. Build the harness (hipcc)

```
./build_harness.sh # auto-picks the lone gemm_*.hpp
./build_harness.sh path/to/gemm_X.hpp gfx942 # explicit header + arch
```

Produces the `harness` binary. Run it on a GPU node:

```
./harness -m=512 -n=512 -k=512 -verify=1
```

4. Full orchestration

CPU box — Stage 1 runs, Stages 2–3 skip:

```
python check_parity.py configs/single_fp16_rcr.json
```

See the entire plan (all three stages) without executing:

```
python check_parity.py configs/single_fp16_rcr.json --dry-run
```

On a GPU node

Dispatcher-only numerical + performance:

```
python check_parity.py configs/single_fp16_rcr.json \
--sizes 512x512x512,1024x1024x1024,2048x2048x2048 \
--arch gfx942
```

Full dispatcher-vs-Tile-Engine (numerical then performance):

```
python check_parity.py configs/single_fp16_rcr.json \
--te-build-dir /path/to/tile_engine/build \
--perf-tol 0.10
```

`--te-build-dir` is searched recursively for `benchmark_gemm_universal_<name>`. Tile Engine writes `latency,tflops,bandwidth` to a CSV (only when it verifies), which the orchestrator parses for both the numerical pass and the performance baseline.

check_parity.py options

Flag	Default	Meaning
`config`	(required)	Tile Engine config JSON (positional)
`--index`	0	Which translated config to check
`--sizes`	512…,1024…,2048…	Comma-separated `MxNxK` problem sizes
`--arch`	gfx942	GPU arch for the harness build
`--te-build-dir`	(none)	Enables dispatcher-vs-TE comparison
`--perf-tol`	0.10	Relative throughput tolerance (10%)
`--output-dir`	generated/	Codegen output directory
`--kernel-set`	parity_single	Kernel-set subdirectory name
`--dry-run`	off	Print the command plan; execute nothing

Reading the result

- **identifier: PASS** — offline and runtime registry keys match.
- **numerical: PASS** — dispatcher (and TE, if given) verified for every size.
- **performance: PASS** — throughput within `--perf-tol`.
- **SKIPPED (no GPU / no hipcc)** — gated, not a failure; exit 0 if Stage 1 passed.
- Per-size **SKIPPED** — the kernel rejected those args; treated as a skip.

Exit code is 0 when nothing failed (skips are OK), non-zero on any real FAIL.

Troubleshooting

Symptom	Cause / fix
`no host C++ compiler found`	Install g++/clang++; needed for the identifier oracle.
Stage 2/3 always SKIPPED	No GPU detected (`rocminfo` shows no `gfx`). Run on a GPU node.
`expected generated header not found`	Codegen failed earlier; re-run drive_codegen.py and read its output.
TE executable not found	Wrong `--te-build-dir`, or that kernel was not built in Tile Engine.
Harness fails at `hipMalloc`	No ROCm device — build is fine, you are on a CPU box.